

The Hierarchical Gaussian Filter

A plain-language guide to how it works

This guide explains the three-level Hierarchical Gaussian Filter (HGF), a model of how an agent learns about an uncertain, changing world. The goal here is understanding, not brevity: every symbol is introduced in words before it appears in an equation, and every equation is followed by an explanation of what it is really doing. We build up from the intuition, through the model of the world, to the learning rules the model produces.

1 What problem does the HGF solve?

Imagine you are playing a game where, on each trial, a light flashes either blue or green, and you have to predict which one will come next. Behind the scenes there is some probability that the light is blue. Sometimes that probability is steady for a long stretch; sometimes it drifts; and occasionally the rules change abruptly and what used to be likely is suddenly rare. You never see the probability directly — you only see the outcomes, one at a time — and yet somehow you have to track it.

This is the situation the HGF is built for. It is a model of an agent that must:

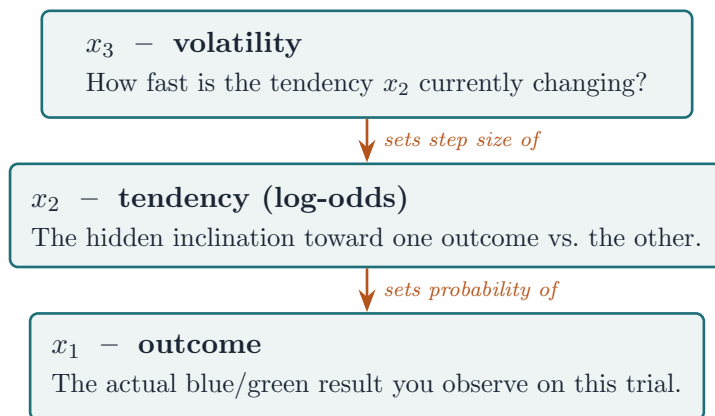
- learn a hidden quantity (here, “how likely is blue?”) from noisy, one-at-a-time observations,
- keep track of *how uncertain* it is about that quantity, not just its best guess,
- and notice when the world has become more volatile, so it knows to learn faster.

That last point is the heart of the model. A good learner does not use a fixed learning rate. In stable times it should be sluggish and ignore the occasional surprise as noise; in turbulent times it should be nimble and update quickly. The HGF achieves this automatically by stacking beliefs into a hierarchy, where a higher level tracks the volatility of the level beneath it.

The HGF is a tower of beliefs in which each level performs a Gaussian random walk, and each level’s current value sets how big the next step of the level below is allowed to be — so “how fast things change” is itself something the agent learns.

2 The core picture: a tower of beliefs

The three-level HGF imagines the world as three hidden quantities, stacked, that change over time (the trials are indexed by k):



Reading from the bottom up:

- x_1 is the outcome itself — a single blue/green event. In the standard “binary” HGF this is a coin flip, a 0 or a 1.
- x_2 is the hidden *tendency* behind those flips. It is a continuous number that lives on the whole real line and represents the *log-odds* of the outcome (we explain this below). Big positive x_2 means “blue is very likely”; big negative means “green is very likely”; zero means “fifty-fifty.”
- x_3 is the *volatility* — how quickly the tendency x_2 is drifting at the moment. When x_3 is high, x_2 can lurch around from trial to trial; when x_3 is low, x_2 is nearly frozen.

The crucial coupling is the downward arrow: *the value of each level sets the step size of the random walk one level below it*. x_3 controls how restless x_2 is, and x_2 controls how the coin at x_1 is biased. This single idea — “the level above sets the volatility of the level below” — is what makes the whole model tick.

3 The generative model: the world the agent imagines

A “generative model” is the agent’s internal story of how the observations are produced. The HGF’s story is three coupled equations, one per level. We write $\mathcal{N}(m, v)$ for a Gaussian (normal) distribution with mean m and variance v .

3.1 Level 3 — the volatility drifts slowly

At the top, volatility itself wanders. From one trial to the next, x_3 takes a small random Gaussian step:

$$x_3^{(k)} \sim \mathcal{N}\left(x_3^{(k-1)}, \vartheta\right).$$

In words: today’s volatility is yesterday’s volatility plus a bit of random drift. The size of that drift is a fixed number ϑ (the Greek letter “theta”), often written as $\vartheta = e^{\omega_3}$. This is a free parameter — a personality trait of the agent — describing how much it expects the volatility to change over time.

3.2 Level 2 — the tendency wanders, but at a speed set from above

The tendency also takes a Gaussian step each trial, but here is the twist: the *size* of its step is not fixed. It is governed by the current volatility x_3 :

$$x_2^{(k)} \sim \mathcal{N}\left(x_2^{(k-1)}, \exp(\kappa x_3^{(k)} + \omega)\right).$$

The variance of the step is $\exp(\kappa x_3 + \omega)$. Let us unpack that exponential, because it carries all the coupling:

- The **exponential** guarantees the variance is always positive (a variance can never be negative), no matter what x_3 does.
- ω (“omega”) is the **tonic volatility**: the baseline, always-on amount of drift at this level, present even when $x_3 = 0$. A larger ω means a fundamentally more changeable tendency.
- κ (“kappa”) is the **coupling strength**: how strongly the upper level’s volatility actually feeds down into level 2. If $\kappa = 0$, level 3 is disconnected and the step size is just the constant e^ω . The larger κ is, the more the agent’s sense of turbulence up top translates into restlessness down here.

When x_3 is high, $\exp(\kappa x_3 + \omega)$ is large, so x_2 is allowed to jump a lot between trials — the agent believes the world is currently unstable. When x_3 is low, that variance shrinks and x_2 barely moves — the agent believes things are settled. The agent does not know x_3 ; it must *infer* it from how surprising the outcomes have been. That inference is what gives the HGF its adaptive learning rate.

3.3 Level 1 — the outcome is a biased coin flip

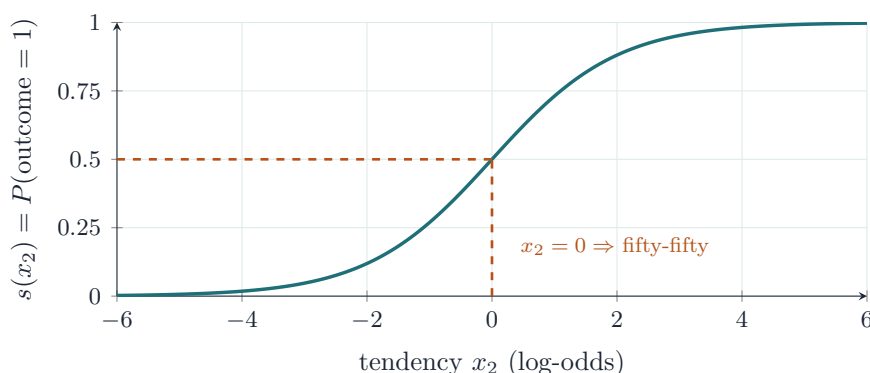
Finally, the observed outcome is generated by flipping a coin whose bias is set by the tendency x_2 . But x_2 can be any real number, while a coin’s probability must sit between 0 and 1. We need something that squashes the whole real line into the interval $(0, 1)$. That something is the **logistic sigmoid**:

$$s(x) = \frac{1}{1 + e^{-x}}.$$

With it, the outcome is a Bernoulli (single coin-flip) random variable:

$$x_1^{(k)} \sim \text{Bernoulli}(s(x_2^{(k)})),$$

meaning the probability of seeing a 1 (say, blue) is $s(x_2^{(k)})$.



The sigmoid turns the log-odds x_2 into a probability. Notice its shape: it is steepest in the middle (around $x_2 = 0$, where the outcome is most uncertain) and flattens out at the edges (where the agent is already nearly certain). That shape, as we will see next, is exactly what controls how much each observation teaches the agent.

4 The Bernoulli part, in depth

The bottom level is where the binary HGF differs from a purely Gaussian model, so it is worth slowing down. Two quantities that appear later in the learning rules — a prediction error and a precision (information) term — both fall directly out of the coin-flip likelihood. Seeing where they come from makes the rest of the model feel inevitable rather than arbitrary.

4.1 Log-odds: what x_2 really means

Saying $p = s(x_2)$ is the same as saying x_2 is the log-odds of the outcome:

$$x_2 = \ln \frac{p}{1-p} \quad \Longleftrightarrow \quad p = s(x_2) = \frac{1}{1 + e^{-x_2}}.$$

Working in log-odds is convenient because it stretches the cramped interval $(0, 1)$ out to the entire real line. A probability of 0.5 becomes $x_2 = 0$; probabilities near 0 or 1 become large negative or positive numbers. This is what lets x_2 do an unbounded Gaussian random walk while the outcome probability stays politely between 0 and 1.

4.2 Where the prediction error and the precision come from

Suppose the agent's current best guess for the tendency is some value of x_2 , and it then observes an outcome $u \in \{0, 1\}$. How well did that value explain what happened? The measure of fit is the **log-likelihood** of the Bernoulli:

$$\ln p(u \mid x_2) = u \ln s(x_2) + (1 - u) \ln(1 - s(x_2)).$$

The learning rules need two things from this: which direction to nudge x_2 , and how confidently. Those are the first and second derivatives.

First derivative — the prediction error. Using the tidy fact that the sigmoid's slope is $s'(x) = s(x)(1 - s(x))$, the first derivative collapses beautifully:

$$\frac{\partial}{\partial x_2} \ln p(u \mid x_2) = u(1 - s) - (1 - u)s = u - s(x_2).$$

So the gradient is simply *outcome minus predicted probability*, $u - s(x_2)$. This is the **prediction error**: the gap between what happened and what the agent expected. If you predicted 0.7 and saw a 1, the error is +0.3 and you should push your belief up; if you saw a 0, it is −0.7 and you push down.

Second derivative — the precision (information). Differentiating once more:

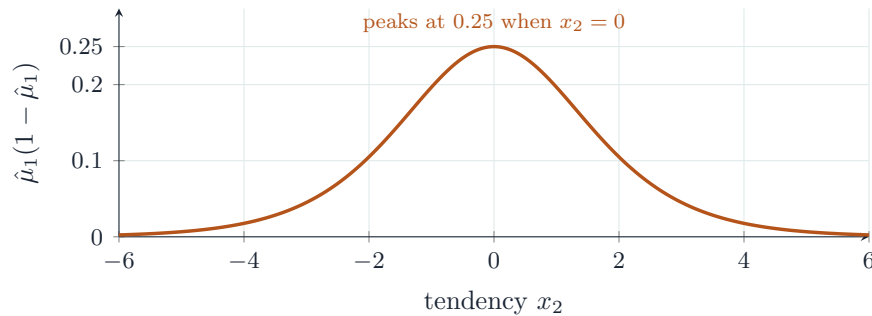
$$\frac{\partial^2}{\partial x_2^2} \ln p(u \mid x_2) = -s(x_2)(1 - s(x_2)).$$

The negative of this curvature, $s(x_2)(1 - s(x_2))$, is how much *information* a single observation carries about x_2 . It is the quantity that will tell the agent how much to trust this observation when updating.

4.3 One term, two meanings

The expression $s(x_2)(1 - s(x_2))$ — writing $\hat{\mu}_1 = s(x_2)$ for the predicted probability, just $\hat{\mu}_1(1 - \hat{\mu}_1)$ — is doing two jobs at once, and the coincidence is the elegant part:

- It is the **variance of the coin flip**, $p(1 - p)$: the irreducible uncertainty in a single Bernoulli outcome. It is largest (0.25) at $p = 0.5$ — a fair coin is maximally unpredictable — and shrinks to zero as p approaches 0 or 1.
- It is also the **slope of the sigmoid** at x_2 : how sensitively the outcome probability responds to a change in the tendency.



Near $x_2 = 0$ the coin is close to fair, the sigmoid is steep, and outcomes are genuinely informative — each result injects a lot of certainty into the agent’s belief, so it learns a lot. Out in the tails, where the agent is already confident, the sigmoid is flat: a confirming outcome barely changes anything, and the information it contributes shrinks toward zero. The same single term governs both the coin’s randomness and the agent’s sensitivity to it.

A note on sensory noise. In the basic binary HGF the outcome is assumed to be observed perfectly, so the agent knows u exactly and the Bernoulli enters *only* through the sigmoid link between x_2 and the outcome. If you add “perceptual uncertainty” (the agent is unsure what it even saw), the observation gets down-weighted by a finite input precision before it reaches level 2, but everything above is unchanged.

5 From the model to learning: inverting the story

So far we have the agent’s *forward* story: volatility shapes the tendency, the tendency biases the coin, the coin produces an outcome. But the agent lives in the opposite direction. It sees outcomes and must work *backwards* to update its beliefs about the hidden tendency and volatility. This backwards step is called **inverting** the model.

Doing this exactly is intractable. The HGF’s contribution is an approximation — based on assuming the beliefs at each level stay approximately Gaussian (a “variational” or Laplace approximation) — that yields remarkably simple, closed-form update rules. On each trial, the agent represents its belief at each level by just two numbers:

- a **mean** μ_i — its best guess for level i , and
- a **precision** $\pi_i = 1/\sigma_i$ — how confident it is (precision is one over the variance; high precision = tight, confident belief).

We use a **hat** to mark a *prediction* — a belief held *before* the new outcome arrives — and no hat for the *posterior*, the updated belief afterward. So $\hat{\mu}_2^{(k)}$ is the predicted mean of the tendency on trial k , and $\mu_2^{(k)}$ is the updated mean once the outcome is in.

Each trial has the same two beats. **Predict:** carry yesterday’s beliefs forward and form an expectation for today. **Update:** see the outcome, measure the surprise, and correct each belief in proportion to how much you trust the news. Every equation below is one of those two beats.

6 The update equations

Here are the actual learning rules for the three-level binary HGF, beat by beat. Parameters: κ (coupling between levels 2 and 3), ω (tonic volatility at level 2), and ϑ (the level-3 drift variance).

6.1 Beat 1 — Predict

Because the random walks have no built-in drift, the predicted mean of each level is just yesterday’s value:

$$\hat{\mu}_2^{(k)} = \mu_2^{(k-1)}, \quad \hat{\mu}_3^{(k)} = \mu_3^{(k-1)}.$$

The predicted *step size* at level 2 — how much the tendency might have moved since last trial — uses the current volatility estimate:

$$\nu_2^{(k)} = \exp(\kappa \mu_3^{(k-1)} + \omega).$$

This ν_2 is the same $\exp(\kappa x_3 + \omega)$ from the generative model, now evaluated at the agent’s best guess of x_3 . The predicted precision at level 2 spreads yesterday’s uncertainty by that step:

$$\hat{\pi}_2^{(k)} = \frac{1}{\sigma_2^{(k-1)} + \nu_2^{(k)}}.$$

The denominator is yesterday’s variance *plus* the expected drift — more expected drift means a less precise (more spread-out) prediction. Finally, the predicted probability of the outcome is the sigmoid of the predicted tendency:

$$\hat{\mu}_1^{(k)} = s(\hat{\mu}_2^{(k)}).$$

6.2 Beat 2, level 2 — update the tendency

The outcome $u^{(k)}$ arrives. The prediction error is exactly the Bernoulli gradient we derived earlier:

$$\delta_1^{(k)} = u^{(k)} - \hat{\mu}_1^{(k)}.$$

The precision of the belief grows by the information the coin flip carried — the Bernoulli variance term, also derived earlier:

$$\pi_2^{(k)} = \hat{\pi}_2^{(k)} + \hat{\mu}_1^{(k)}(1 - \hat{\mu}_1^{(k)}).$$

And the mean is nudged in the direction of the error, scaled by how much the agent trusts the news:

$$\mu_2^{(k)} = \hat{\mu}_2^{(k)} + \frac{1}{\pi_2^{(k)}} \delta_1^{(k)}.$$

Look at the mean update. The change in belief is $\frac{1}{\pi_2} \delta_1$: a prediction error multiplied by a learning rate $1/\pi_2$. The more precise (confident) the belief, the smaller the learning rate, and the less a single surprise shifts it. This (learning rate) $\times$ (prediction error) form is the signature of the HGF, and it is exactly the shape of a reinforcement-learning update — except the learning rate is not fixed.

6.3 Beat 2, level 3 — update the volatility

The top level asks a subtler question: was the tendency x_2 more or less wobbly than I expected? The relevant surprise is therefore a *volatility* prediction error (sometimes “VOPE”), which compares how much x_2 actually moved (and how uncertain it remained) against how much movement was predicted:

$$\delta_2^{(k)} = \frac{\sigma_2^{(k)} + (\mu_2^{(k)} - \hat{\mu}_2^{(k)})^2}{\sigma_2^{(k-1)} + \nu_2^{(k)}} - 1.$$

If the tendency jumped more than expected, the numerator outgrows the denominator, δ_2 is positive, and the agent concludes the world is more volatile than it thought. A handy weighting factor appears repeatedly — the fraction of level-2 uncertainty that is due to the fresh drift rather than old uncertainty:

$$w_2^{(k)} = \nu_2^{(k)} \hat{\pi}_2^{(k)} = \frac{\nu_2^{(k)}}{\sigma_2^{(k-1)} + \nu_2^{(k)}}.$$

The level-3 prediction precision spreads yesterday’s confidence by the top drift ϑ :

$$\hat{\pi}_3^{(k)} = \frac{1}{\sigma_3^{(k-1)} + \vartheta}.$$

Then the volatility belief updates in the same precision-weighted spirit, just with bulkier book-keeping:

$$\begin{aligned} \pi_3^{(k)} &= \hat{\pi}_3^{(k)} + \frac{1}{2} \kappa^2 w_2^{(k)} \left(w_2^{(k)} + (2w_2^{(k)} - 1) \delta_2^{(k)} \right), \\ \mu_3^{(k)} &= \hat{\mu}_3^{(k)} + \frac{1}{2} \frac{\kappa w_2^{(k)}}{\pi_3^{(k)}} \delta_2^{(k)}. \end{aligned}$$

Despite the extra terms, the mean update has the familiar skeleton: a prediction error δ_2 , multiplied by a learning-rate-like factor $\frac{1}{2} \kappa w_2 / \pi_3$. The coupling κ reappears here, governing how strongly a volatility surprise actually moves the top belief.

(Technical aside: in extreme parameter regions this original precision update can occasionally produce a negative π_3 , which is impossible; practical implementations cap it, and a robust reformulation was proposed more recently.)

6.4 Why the loop is self-regulating

Trace what happens after a run of surprises. Big prediction errors δ_2 push μ_3 up. A higher μ_3 enlarges $\nu_2 = \exp(\kappa \mu_3 + \omega)$ on the next trial. A larger ν_2 lowers $\hat{\pi}_2$, which means a smaller π_2 , which means a *larger* learning rate $1/\pi_2$ at level 2. So the agent, having decided the world got volatile, automatically starts learning faster — and when surprises die down, the same machinery slows it back down. The adaptive learning rate is not bolted on; it emerges from the hierarchy.

7 The same idea for continuous outcomes

If the thing being observed is a continuous measurement rather than a yes/no event, only the bottom level changes. The outcome becomes a noisy Gaussian reading of x_1 , and the special Bernoulli term $\hat{\mu}_1(1 - \hat{\mu}_1)$ is replaced by an ordinary input precision $\pi_u = 1/\alpha$ (where α is the measurement-noise variance). Every level above behaves exactly as before — which is the whole point of the design. Because each pair of levels is coupled the same way (the upper one setting the lower one’s step size), the update rules keep an identical form, and the model extends to four, five, or more levels without anything new to derive.

8 Turning beliefs into actions: the response model

To fit the HGF to a real person’s choices, you need to connect its beliefs to observable behavior. A common bridge maps the predicted outcome probability $\hat{\mu}_1$ to the probability of choosing option 1, with a parameter ζ controlling decisiveness:

$$P(\text{choice} = 1) = \frac{(\hat{\mu}_1)^\zeta}{(\hat{\mu}_1)^\zeta + (1 - \hat{\mu}_1)^\zeta}.$$

A large ζ makes the agent commit hard to whichever option it deems more likely; a small ζ makes it choose more randomly. Some variants make this decisiveness itself depend on the estimated volatility (via $\exp(-\mu_3)$), so the agent acts more erratically precisely when it judges the world to be turbulent.

9 Symbol glossary

Symbol	Meaning
k	Trial number (time index).
x_1, x_2, x_3	The three hidden states: outcome, tendency (log-odds), volatility.
$u^{(k)}$	The observed outcome on trial k (here, 0 or 1).
$s(x)$	Logistic sigmoid $1/(1 + e^{-x})$; turns log-odds into a probability.
μ_i	Posterior <i>mean</i> (best guess) at level i .
σ_i, π_i	Posterior variance and precision at level i ; $\pi_i = 1/\sigma_i$.
$\hat{\mu}_i, \hat{\pi}_i$	<i>Predicted</i> (pre-update) mean and precision at level i .
δ_1	Value prediction error, $u - \hat{\mu}_1$: surprise about the outcome.
δ_2	Volatility prediction error: surprise about how much x_2 moved.
ν_2	Predicted step size at level 2, $\exp(\kappa\mu_3 + \omega)$.
w_2	Weighting factor $\nu_2 \hat{\pi}_2$: share of level-2 uncertainty due to fresh drift.
κ	Coupling strength between levels 2 and 3.
ω	Tonic (baseline) volatility at level 2.
ϑ	Variance of the level-3 random walk (top-level drift).
ζ	Decision noise / decisiveness in the response model.

(1) It is a tower where each level sets the volatility of the one below. (2) Every belief update is a precision-weighted prediction error: surprise times a trust-based learning rate. (3) Because the top level tracks volatility, the learning rate adapts on its own — fast in turbulent times, slow in calm ones.